



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-22

今日榜单

- | | | | |
|-------|--|------------|-----|
| 1 | anthropics/financial-services | Python | 3天 |
| <hr/> | | | |
| 2 | agent-substrate/substrate
Agent Substrate: the core system | Go | NEW |
| <hr/> | | | |
| 3 | dream-num/univer
The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relationa... | TypeScript | NEW |
| <hr/> | | | |
| 4 | davila7/claude-code-templates
CLI tool for configuring and monitoring Claude Code | Python | NEW |
| <hr/> | | | |
| 5 | google/ax
Google's open agentic orchestration runtime | Go | NEW |
| <hr/> | | | |
| 6 | mvt-project/mvt
MVT (Mobile Verification Toolkit) helps with conducting forensics of mobile devi... | Python | 2天 |
| <hr/> | | | |
| 7 | superdesigndev/treg
OpenRouter for agent tools. Join community here:https://discord.gg/6mQYYfMAn | Python | NEW |
| <hr/> | | | |
| 8 | browser-use/video-use
Edit videos with coding agents | Python | NEW |
| <hr/> | | | |

#1

Python

连续 3 天

financial-services

Stars **36,118** Forks **5,289** Today **+436**<https://github.com/anthropics/financial-services>

项目简介：这是 Anthropic 官方发布的一套面向金融服务行业的 Claude 参考实现集合，覆盖投行、股票研究、私募股权和财富管理等核心场景。它解决的是金融机构想把 AI 落地到具体工作流时"从零搭建太贵、合规风险难控"的问题，提供了可直接安装的智能体、技能和数据连接器。

核心功能：

- **端到端命名智能体：**如 Pitch Agent（一键生成可比公司、先例交易、LBO 到品牌化路演材料）、Model Builder（在 Excel 中实时构建 DCF/LBO/三表模型）、Earnings Reviewer（财报电话会+文件→模型更新→研报草稿）。
- **垂直技能插件：**按 FSI 细分行业打包的 skills、slash 命令（如 /comps、/dcf、/earnings）和 MCP 数据连接器，可脱离完整智能体单独使用。
- **双形态部署：**同一套系统提示和技能，既可作 Claude Cowork 插件安装，也可通过 Claude Managed Agents API 部署到自有工作流引擎中。
- **合规设计：**所有输出均为"待人工签署"的草稿，不做投资建议、不执行交易、不入账、不批准开户。

适用场景：投行分析师做路演材料、股票研究员跟踪财报、PE 做估值复核、基金行政做 GL 对账和月末结账、合规团队做 KYC 筛查与 LP 对账单审计。适合希望把 AI 嵌入现有金融工作流、但需要严格人工复核环节的机构团队。

技术亮点：

- **"一次编写、两处运行"架构：**Cowork 插件与 Managed Agent 模板共用同一份 prompt/skills，降低维护成本。
- **自包含插件设计：**每个 agent 插件自带依赖技能，安装即用，无需手动拼装。
- **Managed Agent cookbook：**提供 agent.yaml、叶子 worker 子智能体、steering-event 示例及逐 agent 的安全说明，展示多智能体编排的工程化范式。
- **合作方生态：**包含 LSEG、S&P Global 等合作伙伴构建的插件，打通专业金融数据源。

上手建议：若只是试用，从 Cowork 的 Settings → Plugins 粘贴仓库 URL 或上传 plugins/agent-plugins/ 下某个目录的 zip 最快；若用 Claude Code，先装 financial-analysis 核心技能包，再按需装 pitch-agent、gl-reconciler 等。想深入定制或贡献，建议先读 managed-agent-cookbooks/ 里对应 agent 的目录结构，理解 agent.yaml 和子智能体设计，再参考 scripts/ 中的 deploy-managed-agent.sh、validate.py 等工具进行部署与校验。

#2

Go

NEW

substrate

Agent Substrate: the core system

Stars **2,771** Forks **371** Today **+498**

<https://github.com/agent-substrate/substrate>

项目简介：Agent Substrate 是 Google 开源的一个面向自主智能体（Agent）的安全执行运行时，核心目标是解决“如何以极高密度运行海量沙箱”的问题。它将大量大部分时间处于空闲状态的 Agent 应用复用到少量就绪的 Worker 上，从而实现远超传统容器运行时的部署密度。

核心功能：一是 Actor 生命周期管理（创建/销毁、挂起/恢复），支持亚 500ms 的恢复速度和每秒 500+ 次挂起/恢复；二是实时调度，将 Actor 动态分配到 Worker 并路由流量；三是多沙箱技术支持（microVM、gVisor），提供统一的沙箱生命周期操作；四是完整状态快照，跨休眠周期保留内存和文件系统状态；五是基于 Kubernetes 做底层基础设施供给与 Worker 管理。

适用场景：适合需要大规模、高密度运行有状态 Agent 的团队，比如部署 LangChain、ADK、Claude Code 等框架的 Agent，或运行沙箱化 MCP Server。也适用于需要端到端 Agent 部署（含推理、训练、RL 场景）的基础设施团队。

技术亮点：核心洞察是“Agent 大部分时间空闲”，通过 30x+ 超售（oversubscription）把大量 Actor 映射到少量物理 Pod 上。它不重复造轮子，而是复用 Kubernetes 的 Pod 和自动扩缩容能力，在其上叠加 Agent 专属的低延迟调度层，兼顾了基础设施一致性与 Agent 场景的性能需求。

上手建议：项目目前处于早期开发阶段，API 极可能变动，不建议用于生产。可以从 README 中提到的 Counter Demo 入手，在自己的集群里复现“8 个 Pod 复用 250 个有状态 Actor”的演示，理解 Actor Teleport 和状态持久化机制后再深入源码。

#3

TypeScript

NEW

univer

The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relational Tables, and PDF in one runtime.

Stars **15,030** Forks **1,355** Today **+202**

<https://github.com/dream-num/univer>

项目简介：Univer 是一个开源的 Office SDK，让开发者能在自己的产品中嵌入电子表格、文档、演示文稿等办公能力。它解决的核心问题是：不必依赖 Google Sheets 或微软 Office 这类托管服务，也能构建可深度定制的生产力工具。项目定位是“AI Agent 的 Office 基础设施”，强调人类与 AI 可在同一份文件中协作。

核心功能：一是多品类办公组件统一运行时，涵盖表格、文档、幻灯片、看板、关系表等，共享存储与计算架构；二是基于 Canvas 的高性能渲染与插件化架构，可按需组合功能；三是内置公式引擎和统一的 Facade API，浏览器与 Node.js 端行为一致；四是支持跨工具的内容引用与联动更新；五是提供 CLI、Workspace 等配套项目，方便 AI Agent 创建和审阅办公内容。

适用场景：适合 SaaS 产品、内部工具、BI 平台或 AI 应用需要嵌入表格/文档编辑能力的团队。也适合需要在服务端批量处理工作簿、或让 AI Agent 直接操作 Office 文件的场景。典型用户是希望自建办公体验、而非调用第三方托管 API 的开发者。

技术亮点：采用 Canvas 渲染而非 DOM，在大数据量表场景下性能优势明显；插件化 + 命令 + 服务 + Facade API 的分层设计，让扩展和裁剪都很灵活；同一套架构可跑在浏览器和 Node.js，便于前后端复用逻辑；面向 AI Agent 的设计（如 MCP 预览、隔离 worktree）是当前较前沿的方向。

上手建议：先从官网文档和 Showcase 了解能力边界，用官方 preset 快速跑通一个表格或文档示例，再按需引入插件。想深入定制可研究 Facade API 和自定义插件机制。贡献方面建议先读贡献指南、从 issue 或文档改进入手，并注意区分开源版与 Pro 版的授权范围。

#4

Python

NEW

claude-code-templates

CLI tool for configuring and monitoring Claude Code

Stars **30,962** Forks **3,529** Today **+33**

<https://github.com/davila7/claude-code-templates>

项目简介：这是一个为 Anthropic 的 Claude Code 提供开箱即用配置的 CLI 工具，本质上是一个「配置模板市场 + 包管理器」。它解决了 Claude Code 用户每次都要从零手写 agent、命令、hooks、MCP 集成配置的痛点，让开发者能像 `npm install` 一样一键装配完整的 AI 开发工作流。

核心功能：

- **组件化安装：**通过 `--agent`、`--command`、`--setting`、`--hook`、`--mcp` 等参数按需组合安装，也支持交互式浏览。
- **100+ 预置模板：**涵盖开发团队角色（如前端开发者、代码审查员）、测试生成、性能优化、Git 钩子等场景化配置。
- **MCP 外部集成：**内置对 GitHub、Bright Data 等 MCP 服务的接入模板，扩展 Claude Code 的外部能力。
- **Web 浏览界面** (aitmpl.com)：提供可视化探索和一键安装入口，降低发现成本。

适用场景：适合已经使用 Claude Code 进行日常开发的工程师，尤其是希望快速搭建标准化 AI 辅助工作流（如团队统一代码审查、测试生成规范）的个人或团队。也适合想探索 agent/hook/MCP 组合玩法、但不想手写 JSON 配置的实验者。

技术亮点：项目本质上是一个「配置分发层」，把 Claude Code 的配置文件抽象成可组合、可版本化的 npm 包，用 `npx` 实现零安装运行。这种「配置即包」的思路让 AI 工具链的复用和共享变得像前端依赖一样自然，是 AI 开发工具生态里一个有意思的模式创新。

上手建议：直接运行 `npx claude-code-templates@latest` 进入交互式浏览，先装一两个 agent 或 `command` 感受效果；想贡献的话，可以按现有目录结构（如 `development-team/`、`testing/`）提交新的配置模板 PR，门槛不高且社区活跃。

#5

Go

NEW

ax

Google's open agentic orchestration runtime

Stars **7,007** Forks **332** Today **+2,324**<https://github.com/google/ax>

项目简介：AX 是 Google 开源的一个声明式智能体编排运行时，用 Go 编写，定位为“智能体工作负载领域的 Kubernetes”。它解决的核心问题是：AI 智能体既不是无状态微服务，也不是一次性批处理任务，它们会积累状态、需要严格隔离、会调用外部模型和工具，还可能因失控循环而烧钱，传统编排工具难以胜任。AX 通过声明式 YAML 清单和一套小型原语，让这类工作负载可以在集群上规模化、安全地运行。

核心功能：

- **Task（任务）**：在带 CPU/内存限制的沙箱中运行不受信任的智能体代码，支持 `ax ssh` 进入查看、`ax suspend/resume` 暂停与恢复（断点续跑）。
- **Workspace（工作区）**：预置 Git 仓库、MCP 服务器和技能包，让每个智能体启动即“热”状态，无需重复拉取配置。
- **Gateway（网关）**：将出站流量锁定到显式主机白名单，实现网络围栏（network fencing），防止智能体访问未授权地址。
- **Model（模型）**：声明平台自身使用哪个 LLM，凭据从 Kubernetes Secret 读取，统一管理模型接入。
- 全部以 `ax.io/v1alpha1` 清单表达，配合 `ax apply/get/watch/describe/delete` 等 kubectl 风格命令操作。

适用场景：主要面向需要在集群上大规模运行 AI 智能体工作负载的团队，例如自动化代码修复、持续集成中的智能体任务、多智能体流水线等。适合已经熟悉 Kubernetes、希望用类似心智模型来编排智能体的平台工程师和 AI 基础设施团队。当前项目处于早期，核心概念和协议仍在演进，会有破坏性变更，因此更适合探索性、内部实验性场景，而非生产关键路径。

技术亮点：

- 一是**声明式 + 沙箱化**的设计，把智能体当作一等公民工作负载，用清单描述一切并通过 Agent Substrate 提供隔离执行环境；
- 二是**状态可暂停可恢复**，针对智能体“积累状态”的特性设计了 `suspend/resume` 语义，弥补了微服务与批处理模型的空白；
- 三是**网络围栏与预热工作区**，通过 Gateway 白名单和 Workspace 预置解决安全与冷启动两大痛点；
- 四是**kubectl 式的 CLI 与 gRPC 控制面**，降低 K8s 用户的学习成本，并强调单集群可跑“数十亿任务”的高吞吐目标。

上手建议：先从安装 CLI (`go install github.com/google/ax/cmd/ax@latest`) 和阅读 `docs/concepts.md` 开始，理解 Task/Workspace/Gateway/Model 四个原语及任务阶段流转；然后

按 Quick start 部署控制面（需要 K8s 集群、ko 和可达的 Agent Substrate API），跑通 examples/task.yaml，或直接执行 `./demo.sh` 体验完整生命周期。想深入贡献的话，建议阅读 DESIGN.md（架构与 API 参考）和 docs/runner.md（控制面与任务容器的契约），从构建自定义 runner 镜像或参与协议规范讨论入手；由于项目仍在快速迭代，参与前最好先关注其破坏性变更节奏。

#6

Python

连续 2 天

mvt

MVT (Mobile Verification Toolkit) helps with conducting forensics of mobile devices in order to find signs of a potential compromise.

Stars **13,832** Forks **1,342** Today **+441**

<https://github.com/mvt-project/mvt>

项目简介：MVT (Mobile Verification Toolkit) 是由国际特赦组织安全实验室开发的开源移动设备取证工具集，用于检测 Android 和 iOS 设备是否被 Pegasus 等间谍软件感染。它诞生于 2021 年“飞马计划”调查背景，旨在让调查者和技术人员能够自动化地扫描设备中的入侵痕迹。

核心功能： - 支持 iOS 备份与 Android 设备的取证采集，提取短信、通话记录、浏览器历史、已安装应用等关键数据。 - 基于公开的威胁指标 (IOC) 扫描设备，识别已知间谍软件活动的潜在痕迹。 - 提供 mvt-ios、mvt-android 和 mvt 三个命令行工具，分别处理对应平台的分析任务。 - 支持通过插件机制扩展取证模块，并可从 PyPI 安装或通过 uv 作为 CLI 工具使用。

适用场景：主要面向数字取证研究人员、安全调查记者、人权组织技术团队以及参与公民社会数字安全防护的专业人员。典型场景包括：记者或活动人士怀疑设备被监控时的取证排查、间谍软件感染事件的应急响应，以及基于 IOC 的大规模威胁狩猎。

技术亮点：项目以 Python 实现，采用模块化插件架构，便于社区贡献新的取证检查项；v3 版本引入了破坏性变更，说明其正在积极演进 CLI 交互与输出格式。它明确区分“公开 IOC 扫描”与“非公开威胁情报”，在工具定位上保持了取证严谨性，避免用户因仅依赖公开指标而产生虚假安全感。

上手建议：普通用户应先阅读官方文档 (docs.mvt.re) 了解安装依赖和基本命令，从 `pip3 install mvt` 或 `uv tool install mvt` 开始，再尝试对测试设备备份运行 `mvt-ios check-backup` 或 `mvt-android check-adb`。贡献者可从 GitHub Issues 中标记为 good first issue 的任务入手，或参考插件开发文档为项目添加新的取证模块。需注意该工具面向专业取证人员，不建议普通用户用于自我安全评估。

#7

Python

NEW

treg

OpenRouter for agent tools. Join community here: <https://discord.gg/6mQYyFMAn>

Stars **2,032** Forks **207** Today **+197**

<https://github.com/superdesigndev/treg>

项目简介：Treg 是一个面向 AI Agent 的"工具聚合网关"，定位类似 OpenRouter，但聚合的对象是工具而非大模型。它把 Semrush、Moz、Crunchbase、Apollo 等原本需要单独订阅、注册甚至邀请才能使用的付费工具，统一到一个 base URL 和一个 token 之下，让 Agent 按调用付费、无需逐个开通账号。

核心功能：

- **工具目录 (Catalog)：**收录 60+ 供应商、3000+ 端点，覆盖 SEO/外链、社交趋势、人物与公司信息富集、广告、爬虫、图像视频生成等，按次计价（低至几美分）。
- **自带工具注册：**团队成员可注册自己的付费 API、OAuth 连接、厂商 CLI 或 SKILL.md 技能包，自有密钥优先且不计费。
- **统一鉴权代理：**服务端注入认证信息，调用方永远不接触密钥，且代理只做"转发"不做"建模"，从而抵御上游 API 变更。
- **CLI 与插件集成：**提供 treg 命令行工具，并支持作为 Claude Code 插件、MCP 连接器（/mcp/v2/）接入各类 Agent。

适用场景：适合需要让 Agent 完成真实商业任务（查外链、做公司富集、跑趋势分析）但不想为单次运行购买昂贵订阅的团队；也适合希望把内部密钥/CLI/技能安全共享给所有同事 Agent 而不泄露凭证的组织。

技术亮点：核心设计是"relays, never models"——代理只转发不解析上游结构，因此上游 API 变化时无需改动；认证在服务端通过 binding 机制注入（一次请求可携带多个密钥，如 OAuth bearer + developer-token 头）；MCP 层面还区分读写调用以给 Claude 提供准确的安全信号。

上手建议：最快路径是执行 `curl -fsSL https://treg.to/install.sh | sh` 安装 CLI，然后 `treg login` 登录，用 `treg catalog search` 按"要做什么"而非"用哪个工具"来检索，再用 `treg call` 直接调用。开发者也可通过 Claude Code 插件市场或 `npx skills add` 集成；项目支持自托管，想深入贡献可从 README 中的架构文档（MCP/OAuth）和 docs/ 目录入手。

#8

Python

NEW

video-use

Edit videos with coding agents

Stars **25,593** Forks **3,100** Today **+155**<https://github.com/browser-use/video-use>

项目简介

video-use 是 browser-use 团队推出的开源视频编辑工具，核心理念是"用 Claude Code 剪视频"。用户只需把原始素材丢进文件夹，用自然语言对话（比如"把这些剪成一支发布视频"），就能得到 final.mp4，无需预设模板或菜单操作。

核心功能

- **智能剪辑**：自动剔除"嗯""呃"等填充词、口误和无效停顿，实现词级精度的切割。
- **自动调色与音频处理**：为每个片段自动调色（暖色电影感、中性锐利或自定义 ffmpeg 链），并在每个剪切点加 30ms 音频淡入淡出，避免爆音。
- **字幕与动画叠加**：按用户风格烧录字幕（默认两词大写块），并通过 HyperFrames、Remotion、Manim 或 PIL 并行生成动画叠加层。
- **自评估渲染**：在每个剪切边界对渲染结果做自检，捕捉画面跳变、音频爆音、隐藏字幕等问题，出错则修复重渲（最多 3 次）。
- **会话记忆持久化**：通过 project.md 保存上下文，下次打开能接着上次的进度继续。

适用场景

适合内容创作者、播客/访谈剪辑者、教程制作者，以及任何需要快速把口播、旅行、访谈等多段素材剪成成片的人。尤其适合已在用 Claude Code / Codex 等 coding agent、希望把视频剪辑也纳入"对话式工作流"的用户。

技术亮点

最巧妙的一点是 **LLM 从不"看"视频，而是"读"视频**。它用两层表示：一是 ElevenLabs Scribe 生成的词级时间戳转写（打包成约 12KB 的 takes_packed.md），二是按需调用的 timeline_view 视觉合成图（胶片条+波形+词标签）。相比"3 万帧 × 1500 token = 4500 万 token 噪声"的朴素方案，这

套方法把信息压缩到 12KB 文本加少量 PNG，思路与 browser-use 用结构化 DOM 替代截图如出一辙。
整个流水线为：转写 → 打包 → LLM 推理 → 生成 EDL → 渲染 → 自评估。

上手建议

最省事的方式是把 README 里的 setup prompt 直接粘进 Claude Code、Codex 等有 shell 权限的 agent，让它自动完成 clone、依赖安装、skill 注册，并提示你输入 ElevenLabs API key。想手动安装的话，按 README 执行 `git clone + uv sync + brew install ffmpeg`，再配置 `.env` 即可。想贡献代码，建议先读 `install.md`、`SKILL.md` 和 `helpers/` 目录（编辑脚本都在那里），从理解 EDL 生成和自评估循环入手。

数据来源: github.com/trending

报告日期: 2026-09-22

由 DeepSeek AI 提供智能分析 | 自动生成